Question 1: Exception Handling - Robust Calculator

↳ 1 cell hidden

Question 2: File Operations - Text File Manager

↳ 1 cell hidden

Question 3: CSV File Processing with Error Handling

↳ 2 cells hidden

Question 4: JSON Configuration Manager

↳ 1 cell hidden

Question 5: Logging System with File Management

↳ 1 cell hidden

Question 6: Data Backup System

↳ 1 cell hidden

Question 7: File Validator and Processor

```
import os
from datetime import datetime

class InvalidFileExtensionError(Exception):
    pass


class FileSizeExceededError(Exception):
    pass


class InvalidFileFormatError(Exception):
    pass


os.makedirs("data", exist_ok=True)

# Valid TXT
with open("data/file1.txt", "w") as f:
    f.write("This is a valid text file.")

# Valid CSV
with open("data/file2.csv", "w") as f:
    f.write("id,name\n1,John")

# Invalid Extension
with open("data/file3.invalid", "w") as f:
    f.write("Invalid extension")

# Large File (for testing)
with open("data/large_file.txt", "w") as f:
    f.write("A" * 20000)      # 20 KB
```

```
print("Sample files created.\n")

DATA_DIR = "data"
ERROR_REPORT = "error_report.txt"

SUPPORTED_EXTENSIONS = {
    ".txt",
    ".csv"
}

MAX_FILE_SIZE = 10000   # 10 KB

error_log = []


def log_error(file_name, error_type, message):

    timestamp = datetime.now().strftime(
        "%Y-%m-%d %H:%M:%S"
    )

    error_log.append(
        f"[{timestamp}] "
        f"{file_name} | "
        f"{error_type} | "
        f"{message}"
    )


def validate_file(file_path):

    # File Exists

    if not os.path.exists(file_path):

        raise FileNotFoundError(
            "File does not exist."
        )

    # Read Permission

    if not os.access(file_path, os.R_OK):

        raise PermissionError(
            "Read permission denied."
        )

    # Extension Check

    extension = os.path.splitext(
        file_path
    )[1].lower()

    if extension not in SUPPORTED_EXTENSIONS:

        raise InvalidFileExtensionError(
            f"Extension '{extension}' "
            f"not supported."
        )

    # File Size Check

    size = os.path.getsize(file_path)

    if size > MAX_FILE_SIZE:

        raise FileSizeExceededError(
            f"File size ({size} bytes) "
            f"exceeds limit "
            f"({MAX_FILE_SIZE} bytes)"
        )

    # Encoding Check

    try:

        with open(
            file_path,
```

```
            "r",
            encoding="utf-8"
        ) as file:

            file.read()

    except UnicodeDecodeError:

        raise InvalidFileFormatError(
            "Invalid UTF-8 encoding."
        )

    return True

def process_file(file_path):

    with open(
        file_path,
        "r",
        encoding="utf-8"
    ) as file:

        content = file.read()

    return {
        "characters": len(content),
        "lines": len(content.splitlines())
    }


print(
    "=== File Validator and Processor ==="
)

print(
    f"\nProcessing files in "
    f"directory: ./{DATA_DIR}/"
)

total_files = 0
processed_files = 0
failed_files = 0

for file_name in os.listdir(DATA_DIR):

    file_path = os.path.join(
        DATA_DIR,
        file_name
    )

    total_files += 1

    print(
        f"\nProcessing {file_name}..."
    )

    try:

        validate_file(file_path)

        result = process_file(
            file_path
        )

        print(
            "✓ Validated and "
            "processed successfully"
        )

        print(
            f"Characters: "
            f"{result['characters']}"
        )

        print(
            f"Lines: "
            f"{result['lines']}"
```

```
        )

        processed_files += 1

    except InvalidFileExtensionError as e:

        print(
            f"✗ Error: "
            f"InvalidFileExtensionError - "
            f"{e}"
        )

        log_error(
            file_name,
            "Extension Error",
            str(e)
        )

        failed_files += 1

    except FileSizeExceededError as e:

        print(
            f"✗ Error: "
            f"FileSizeExceededError - "
            f"{e}"
        )

        log_error(
            file_name,
            "Size Error",
            str(e)
        )

        failed_files += 1

    except InvalidFileFormatError as e:

        print(
            f"✗ Error: "
            f"InvalidFileFormatError - "
            f"{e}"
        )

        log_error(
            file_name,
            "Format Error",
            str(e)
        )

        failed_files += 1

    except PermissionError as e:

        print(
            f"✗ Error: "
            f"PermissionError - "
            f"{e}"
        )

        log_error(
            file_name,
            "Permission Error",
            str(e)
        )

        failed_files += 1

    except FileNotFoundError as e:

        print(
            f"✗ Error: "
            f"FileNotFoundError - "
            f"{e}"
        )

        log_error(
            file_name
```

```
            "File Not Found",
            str(e)
        )

        failed_files += 1

    except Exception as e:

        print(
            f"✗ Unexpected Error: "
            f"{e}"
        )

        log_error(
            file_name,
            "Unexpected Error",
            str(e)
        )

        failed_files += 1


with open(
    ERROR_REPORT,
    "w"
) as report:

    report.write(
        "=== Error Report ===\n\n"
    )

    for error in error_log:

        report.write(
            error + "\n"
        )


print(
    "\n=== Processing Summary ==="
)

print(
    f"Total Files: {total_files}"
)

print(
    f"Processed: {processed_files}"
)

print(
    f"Failed: {failed_files}"
)

print(
    f"Errors logged to: "
    f"{ERROR_REPORT}"
)
```

```
Sample files created.

=== File Validator and Processor ===

Processing files in directory: ./data/

Processing large_file.txt...
✗ Error: FileSizeExceededError - File size (20000 bytes) exceeds limit (10000 bytes)

Processing file2.csv...
✓ Validated and processed successfully
Characters: 14
Lines: 2

Processing file3.invalid...
✗ Error: InvalidFileExtensionError - Extension '.invalid' not supported.

Processing file1.txt...
✓ Validated and processed successfully
```

```
Characters: 26
Lines: 1

=== Processing Summary ===
Total Files: 4
Processed: 2
Failed: 2
Errors logged to: error_report.txt
```

Question 8: Comprehensive File Management System

```
import os
import csv
import json
import shutil
from datetime import datetime

os.makedirs("backups", exist_ok=True)

ERROR_LOG = "error_log.txt"
OPERATION_LOG = "operations.log"

SUPPORTED_EXTENSIONS = {
    ".txt",
    ".csv",
    ".json"
}

locked_files = set()

with open("employees.csv", "w", newline="") as file:

    writer = csv.writer(file)

    writer.writerow(
        ["name", "age", "email", "salary"]
    )

    writer.writerow(
        ["John Doe", 30,
         "john@example.com", 50000]
    )

    writer.writerow(
        ["Jane Smith", 25,
         "jane@example.com", 45000]
    )

    writer.writerow(
        ["Bob Johnson", 35,
         "bob@example.com", 60000]
    )

with open("config.json", "w") as file:

    json.dump(
        {
            "app_name": "My Application",
            "version": "1.0.0",
            "settings": {
                "debug": False,
                "timeout": 30
            }
        },
        file,
        indent=4
    )

def log_error(message):

    with open(
        ERROR_LOG,
        "a"
    ) as file:
```

```
            file.write(
                f"[{datetime.now()}] "
                f"{message}\n"
            )


    def log_operation(message):

        with open(
            OPERATION_LOG,
            "a"
        ) as file:

            file.write(
                f"[{datetime.now()}] "
                f"{message}\n"
            )

    def validate_file(filename):

        extension = os.path.splitext(
            filename
        )[1].lower()

        if extension not in SUPPORTED_EXTENSIONS:

            raise ValueError(
                f"Unsupported format: "
                f"{extension}"
            )

    def create_backup(filename):

        if not os.path.exists(filename):
            return

        timestamp = datetime.now().strftime(
            "%Y%m%d_%H%M%S"
        )

        backup_name = (
            f"backups/"
            f"{timestamp}_"
            f"{os.path.basename(filename)}"
        )

        shutil.copy2(
            filename,
            backup_name
        )

    def lock_file(filename):

        if filename in locked_files:

            raise RuntimeError(
                "File is currently locked."
            )

        locked_files.add(filename)


    def unlock_file(filename):

        locked_files.discard(
            filename
        )


    def create_file():

        try:

            filename = input(
                "Enter filename: "
            )
```

```
            validate_file(filename)

            print(
                "Enter content "
                "(END to finish):"
            )

            lines = []

            while True:

                line = input()

                if line == "END":
                    break

                lines.append(line)

            with open(
                filename,
                "w"
            ) as file:

                file.write(
                    "\n".join(lines)
                )

            print(
                "File created successfully."
            )

            log_operation(
                f"Created {filename}"
            )

        except Exception as e:

            print(
                f"Error: {e}"
            )

            log_error(str(e))

    def read_file():

        try:

            filename = input(
                "Enter filename: "
            )

            validate_file(filename)

            with open(
                filename,
                "r"
            ) as file:

                print(
                    "\n=== File Content ==="
                )

                print(
                    file.read()
                )

            log_operation(
                f"Read {filename}"
            )

        except Exception as e:

            print(
                f"Error: {e}"
            )

            log_error(str(e))
```

```
def update_file():

    filename = None

    try:

        filename = input(
            "Enter filename: "
        )

        validate_file(filename)

        lock_file(filename)

        create_backup(filename)

        print(
            "Enter new content "
            "(END to finish):"
        )

        lines = []

        while True:

            line = input()

            if line == "END":
                break

            lines.append(line)

        with open(
            filename,
            "w"
        ) as file:

            file.write(
                "\n".join(lines)
            )

        print(
            "File updated successfully."
        )

        log_operation(
            f"Updated {filename}"
        )

    except Exception as e:

        print(
            f"Error: {e}"
        )

        log_error(str(e))

    finally:

        if filename:
            unlock_file(filename)

def delete_file():

    try:

        filename = input(
            "Enter filename: "
        )

        create_backup(filename)

        os.remove(filename)

        print(
            "File deleted successfully."
```

```
            )

            log_operation(
                f"Deleted {filename}"
            )

        except Exception as e:

            print(
                f"Error: {e}"
            )

            log_error(str(e))

    def list_files():

        print(
            "\n=== Files ==="
        )

        for file in os.listdir():

            if os.path.isfile(file):
                print(file)

    def search_files():

        keyword = input(
            "Enter keyword: "
        )

        print(
            "\nMatching Files:"
        )

        for file in os.listdir():

            if not os.path.isfile(file):
                continue

            try:

                with open(
                    file,
                    "r",
                    encoding="utf-8"
                ) as f:

                    content = f.read()

                    if keyword in content:

                        print(file)

            except:
                pass

    def file_statistics():

        try:

            filename = input(
                "Enter filename: "
            )

            with open(
                filename,
                "r"
            ) as file:

                content = file.read()

            stats = os.stat(filename)

            print(
                "\n=== File Statistics ==="
            )
```

```
            print(
                f"Size: "
                f"{stats.st_size} bytes"
            )

            print(
                f"Lines: "
                f"{len(content.splitlines())}"
            )

            print(
                f"Words: "
                f"{len(content.split())}"
            )

            print(
                f"Characters: "
                f"{len(content)}"
            )

            print(
                "Created:",
                datetime.fromtimestamp(
                    stats.st_ctime
                )
            )

            print(
                "Modified:",
                datetime.fromtimestamp(
                    stats.st_mtime
                )
            )

        except Exception as e:

            print(
                f"Error: {e}"
            )

            log_error(str(e))

    while True:

        print(
            "\n=== File Management System ==="
        )

        print("1. Create File")
        print("2. Read File")
        print("3. Update File")
        print("4. Delete File")
        print("5. List Files")
        print("6. Search Files")
        print("7. File Statistics")
        print("8. Exit")

        choice = input(
            "Enter choice: "
        )

        if choice == "1":
            create_file()

        elif choice == "2":
            read_file()

        elif choice == "3":
            update_file()

        elif choice == "4":
            delete_file()

        elif choice == "5":
            list_files()
```

```
    elif choice == "6":
        search_files()

    elif choice == "7":
        file_statistics()

    elif choice == "8":

        print(
            "Exiting..."
        )

        break

    else:

        print(
            "Invalid choice."
        )
```

```
=== File Management System ===
1. Create File
2. Read File
3. Update File
4. Delete File
5. List Files
6. Search Files
7. File Statistics
8. Exit
Enter choice: 1
Enter filename: test.txt
Enter content (END to finish):
END
File created successfully.

=== File Management System ===
1. Create File
2. Read File
3. Update File
4. Delete File
5. List Files
6. Search Files
7. File Statistics
8. Exit
Enter choice: 2
Enter filename: test.txt

=== File Content ===


=== File Management System ===
1. Create File
2. Read File
3. Update File
4. Delete File
5. List Files
6. Search Files
7. File Statistics
8. Exit
Enter choice: 7
Enter filename: test.txt

=== File Statistics ===
Size: 0 bytes
Lines: 0
Words: 0
Characters: 0
Created: 2026-06-09 10:44:25.258973
Modified: 2026-06-09 10:44:25.258973

=== File Management System ===
1. Create File
2. Read File
3. Update File
4. Delete File
5. List Files
6. Search Files
```